

CS 112

Introduction to Data Structures

WEEK 01

Control Structures, Functions, Parameter Passing

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 Selection: `if` and `switch`
- 2 Loops: `while` , `for` , `break` / `continue`
- 3 Defining and calling functions
- 4 Prototypes, default arguments, overloading
- 5 Arrays as parameters
- 6 Parameter passing: in, out, in/out **KEY**
- 7 Building with `make`

Selection: if

```
if (hp <= 0) {  
    cout << "You have fallen." << endl;  
} else if (hp < 20) {  
    cout << "You are badly hurt." << endl;  
} else {  
    cout << "You fight on." << endl;  
}
```

- Braces are optional for one statement — **use them anyway**
- Boolean operators: `==` `!=` `&&` `||` `!`
- `=` is assignment, `==` is comparison — the classic C++ bug

Format as you go: in VS Code, **F1 → Format Document**. Readable code is hospitality toward whoever reads it next.

Selection: switch

```
switch (roll) {  
    case 1:  
        cout << "Critical miss!" << endl;  
        break;  
    case 19:  
    case 20:                // 19 and 20 share a case  
        cout << "Critical hit!" << endl;  
        break;  
    default:  
        cout << "A normal swing." << endl;  
}
```

Forget break and execution *falls through* into the next case. Sometimes useful, usually a bug.

What prints when roll is 19?

```
case 19:  
case 20:  
    cout << "Critical hit!" << endl;  
    break;  
default:  
    cout << "A normal swing." << endl;
```

- A.** Critical hit!
- B.** Critical hit! then A normal swing.
- C.** A normal swing.
- D.** Nothing — case 19 is empty.

Loops: while

```
// how many times can we halve this before reaching 1?  
double value = 45444443442345345.0;  
int count = 0;  
  
while (value > 1.0) {  
    value /= 2.0;  
    ++count;  
}  
cout << "2^" << count << " clears it" << endl;
```

A while loop tests *before* each pass. If the condition starts false, the body never runs.

What does arr hold after this runs?

```
int arr[] = { 3, 7, 11, 13 };  
int i = 1;  
while (i < 4) {  
    arr[i] = arr[i] * 2;  
    ++i;  
}
```

A. 6, 14, 22, 26

B. 3, 14, 22, 26

C. 6, 14, 22, 13

D. 3, 14, 22, 13

E. None of the others

From while to for

```
int i = 1;
while (i < 4) {
    arr[i] = arr[i] * 2;
    ++i;
}
```

Three things scattered across four lines: **start**, **test**, **step**.

From while to for

```
for (int i = 1; i < 4; ++i) {  
    arr[i] = arr[i] * 2;  
}
```

Same loop, one line of bookkeeping. `i` now lives *only* inside the loop — using it afterwards is a compile error.

break and continue

break — leave the loop

```
for (int i = 0; i < n; ++i) {  
    if (party[i] == target) {  
        found = i;  
        break;           // stop looking  
    }  
}
```

continue — skip this pass

```
for (int i = 0; i < n; ++i) {  
    if (hp[i] <= 0) {  
        continue;       // skip the fallen  
    }  
    total += hp[i];  
}
```

Both are shortcuts, not magic: anything they do can be written with a richer condition. Use them when they make the intent *clearer*.

Defining a Function

```
int rollDamage(int dice, int sides) {  
    int total = 0;  
    for (int i = 0; i < dice; ++i) {  
        total += rand() % sides + 1;  
    }  
    return total;  
}  
  
int main() {  
    int hit = rollDamage(2, 6);    // 2d6  
}
```

Parameters (line 1) — names with types, comma separated.

Arguments (line 11) — the actual values you pass in.

Define Before You Call

C++ reads top to bottom. Calling a function the compiler hasn't seen yet is an error — so we declare it first with a **prototype**.

dice.h

```
int rollDamage(int dice, int sides);
```

Just the signature — *what* exists.

dice.cpp

```
#include "dice.h"

int rollDamage(int dice, int sides) {
    ...
}
```

The body — *how* it works.

Every .cpp that calls it just does `#include "dice.h"`. This is the header/source split from week 0, now with a reason.

Default Arguments & Overloading

```
// default: callers may omit sides
int rollDamage(int dice, int sides = 6);

rollDamage(2);           // 2d6
rollDamage(2, 20);      // 2d20

// overloading: same name, different parameter lists
int  attack(int strength);
int  attack(int strength, int bonus);
void attack(const string &spell);
```

- Defaults go on the **prototype**, not the definition
- Overloads must differ in **parameters** — return type alone is not enough
- The compiler picks the match; ambiguity is a compile error

Passing Arrays

```
void printParty(int hp[], int n);    // these two are
void printParty(int *hp, int n);    // the same thing

int party[] = { 12, 30, 8 };
printParty(party, 3);               // nothing is copied
```

0x7ffd10	12	30	8	← party
hp (in function)	0x7ffd10	← just the address		

An array argument decays to a **pointer to its first element**. The function gets an address, not a copy — which is why it must also be told the size.

Which call passes vals more efficiently?

```
void print(int *arr, int size);  
void print2(int arr[], int size);  
  
int vals[] = { 1, 3, 4 };  
print(vals, 3);  
print2(vals, 3);
```

- A. `print()` is more efficient
- B. `print2()` is more efficient
- C. Neither — they are the same

Will the second call to `print2()` compile?

```
void print2(int arr[], int size);  
  
int vals[] = { 1, 3, 4 };  
int val = 7;  
print2(vals, 3);  
print2(&val, 1);
```

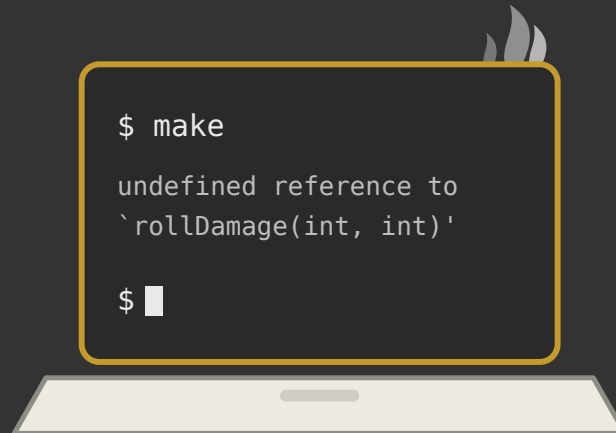
- A. Yes — `&val` is an address, and `int arr[]` takes an address
- B. No — `print2()` requires an actual array
- C. No — you cannot have an array of size 1
- D. None of the above

Protecting an Array: const

```
int totalHP(const int hp[], int size) {  
    int total = 0;  
    for (int i = 0; i < size; ++i) {  
        total += hp[i];  
    }  
    // hp[0] = 99;    □ compile error – hp is const here  
    return total;  
}
```

Marking a parameter const is a promise to the caller: *this function will not modify your data*. The compiler enforces the promise.

Demo



Live coding. What could possibly go wrong?

(the semicolon is always missing on line 12)

Which Way Does the Data Flow?

IN — the function reads it, caller's value never changes

OUT — the function fills it in; whatever was there is ignored

IN/OUT — the function reads it *and* updates it

```
// IN
int totalHP(const int hp[], int n);

// OUT
void rollStats(int &str, int &dex);

// IN/OUT
void applyDamage(int &hp, int hit);
```

Decide the *direction* first. The passing mechanism follows from it.

Three Ways to Pass

Mechanism	Syntax	Copies?	Can change caller's value?
By value	<code>int hp</code>	Yes	No
By reference	<code>int &hp</code>	No	Yes
By const reference	<code>const string &name</code>	No	No

Small and IN? Pass by value — copying an `int` is free.

Big and IN? Pass by const reference — no copy, still safe.

Anything OUT or IN/OUT must be by **plain reference** — that's the only way the caller sees the change.

Does x flow *in*, *out*, or *in/out* of j()?

```
int j(int val) {  
    val = val + 1;  
    return val;  
}  
  
int main() {  
    int x = 3;  
    int y = j(x);  
}
```

- A. In
- B. In/Out
- C. Out
- D. Out, because of what j() returns

Does *x* flow *in*, *out*, or *in/out* of *j()*?

```
void j(int &val) {  
    val = val + 1;  
}  
  
int main() {  
    int x = 3;  
    j(x);  
}
```

- A. In
- B. In/Out
- C. Out
- D. Out, because of what *j()* returns

Does *x* flow *in*, *out*, or *in/out* of *j()*?

```
void j(int &val) {  
    val = 17;  
}  
  
int main() {  
    int x = 3;  
    j(x);  
}
```

- A. In
- B. In/Out
- C. Out
- D. Out, because of what *j()* returns

Building with make

Three files, one command. `make` recompiles only what changed.

```
CXX      = g++
CXXFLAGS = -Wall -std=c++17

game: main.o dice.o
    $(CXX) $(CXXFLAGS) -o game main.o dice.o

main.o: main.cpp dice.h
    $(CXX) $(CXXFLAGS) -c main.cpp

dice.o: dice.cpp dice.h
    $(CXX) $(CXXFLAGS) -c dice.cpp

clean:
    rm -f game *.o
```

Those indents must be **real tabs**, not spaces — the one rule that bites everyone once.

Why make Earns Its Keep

- A target is **rebuilt only if** something it depends on is newer
- Change `dice.cpp` → only `dice.o` recompiles, then it relinks
- `make clean` throws away everything built, so you can start fresh
- Your whole build recipe lives in the repo, not in someone's memory

On a three-file project this saves seconds. On a big one it saves your afternoon — and it's how the autograder builds your code.

Week 01 Recap

- `if`/`switch` and loops read the same as Python — the braces and types are new
- A function must be **declared before it is called** — that's what headers are for
- Arrays are passed as **addresses**, so functions need the size too
- Choose the direction (in / out / in/out) first, then the mechanism
- `make` rebuilds only what changed — and tabs matter

Next: a01 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

